

Finzo: AI Engineering Project Report

Date: Oct 4, 2026

Field	Detail
Name	Ritesh Gawade
Roll No.	25P0620014
Course	AI Core Research 2 - AI Engineering
Project	Finzo, a personal expense tracker built with an AI copilot

Finzo is a server-rendered expense tracker built with FastAPI, Jinja2 and MySQL. This report explains what was built, how it was built with an AI copilot, and how the code quality was checked.

1. Use Case Definition

1.1 Problem Statement

People who track spending in notes or spreadsheets may find it slow to record every expense. It can also be difficult to see totals by category and by time period. Finzo solves this problem for individuals.

A user can create an account, sign in, and manage their own expense records. The profile page shows account details, a spending summary, category totals, and a list of transactions for a selected date range. The interface uses Indian rupees (₹).

Finzo is **not an AI-powered application**. No AI model runs inside the product. AI is used as a development tool. The AI copilot helped write specifications, code, tests, and reviews. Project rules were stored in files so that the copilot could give consistent output. This speeds up routine development work, while the specifications, tests, and reviews help keep the result under control.

The technology stack fits the problem. FastAPI handles HTTP requests, Jinja2 renders pages on the server, and PyMySQL stores user and expense data. The project instructions require direct, parameterized SQL and a split between routes, services, and repositories instead of an ORM.

1.2 Scope

In scope

- Public landing, Terms and Privacy pages.
- Registration, login, logout and signed-session authentication.
- An authenticated profile with user-specific totals, category breakdown, recent activity, date presets and custom date filtering.
- Authenticated add, edit and single-record delete of expenses, limited to the current user.
- An authenticated Analytics page that shows a 'Coming Soon' placeholder.
- A database-backed `/health` endpoint and a Docker image definition.

Out of scope

- Real analytics charts, reports or predictions. The Analytics spec limits the page to a placeholder.
- Budget management, recurring-payment automation, bank synchronization and AI-generated advice.
- Profile editing, bulk delete, undo or restore, and a JSON API.
- Cloud deployment, image publishing and automatic deployment.

1.3 Features

Feature	Input	Output	Status
Public pages	HTTP request	Landing, Terms or Privacy HTML	Implemented
Registration	Name, email, password	Validated account record; redirect to login	Implemented
Login and logout	Email and password; logout as POST	Signed session set or cleared	Implemented
Profile and date filter	Signed-in session; optional date bounds	Profile, spending summary, category totals, transactions	Implemented
Add expense	Title, amount, category, description	New expense for the signed-in user	Implemented
Edit expense	Expense ID and edited fields	Updated expense owned by the user	Implemented
Delete expense	Expense ID and confirmation POST	Expense removed; redirect to profile	Implemented
Analytics	Signed-in session	'Advanced Analytics: Coming Soon' page	Partial
Health check	HTTP request	JSON success, or a generic unavailable response, based on MySQL connectivity	Implemented
Container packaging	Docker build context and environment settings	Non-root FastAPI container definition with health check	Partial

Partial for Analytics means only the signed-in placeholder page exists.

Partial for container packaging means a Dockerfile and ignore rules exist, but no image build or container smoke run was recorded.

1.4 Non-functional Requirements

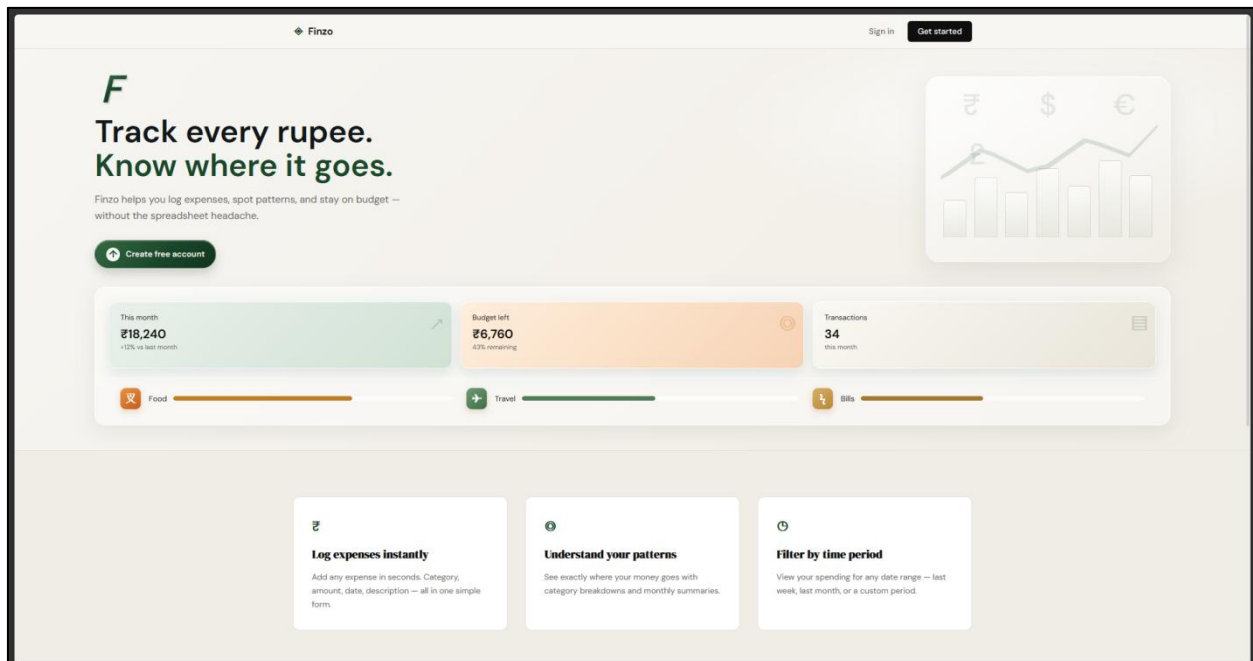
- **Maintainability:** Routes, services, repositories, database helpers, schemas and templates are separate. SQL is parameterized.
- **Security:** Passwords are hashed with Werkzeug. Sessions use Starlette `SessionMiddleware`. Update and delete queries include the signed-in user ID. The cookie uses `SameSite=Lax`, and its `Secure` flag is set by an environment variable.

- **Configuration:** The database URL and session signing key come from the environment. Startup checks required settings. Production does not initialize or seed the database.
- **Usability:** Shared Jinja templates and CSS, labelled forms with feedback, accessible table markup, and responsive breakpoints at 900px and 600px.
- **Testing:** pytest covers routes, validation, ownership, database helpers, health and Docker configuration.
- **Deployment readiness:** The Dockerfile has a configurable port, a non-root user and a health check. No numeric performance target exists.

2. Application Functionality and Build Quality

2.1 Working Functionality

The figures below follow the main user flow, from the public page to deleting an expense.



[Figure 1: Landing page.]

The landing page shows the sign-in and 'Get started' actions and the three feature cards. The sample spending numbers and the chart are static design content, not live data.

The screenshot shows the 'Create your account' page of the Finzo application. The header includes the Finzo logo on the left and 'Sign in' and 'Get started' links on the right. The main heading is 'Create your account' with the subtext 'Start tracking your expenses today'. The registration form is centered and contains three input fields: 'Full name' (filled with 'Ritesh Gawade'), 'Email address' (filled with 'r@gmail.com'), and 'Password' (filled with eight dots). Below these fields is a green 'Create account' button. At the bottom of the form, there is a link that says 'Already have an account? Sign in'. The footer features the Finzo logo.

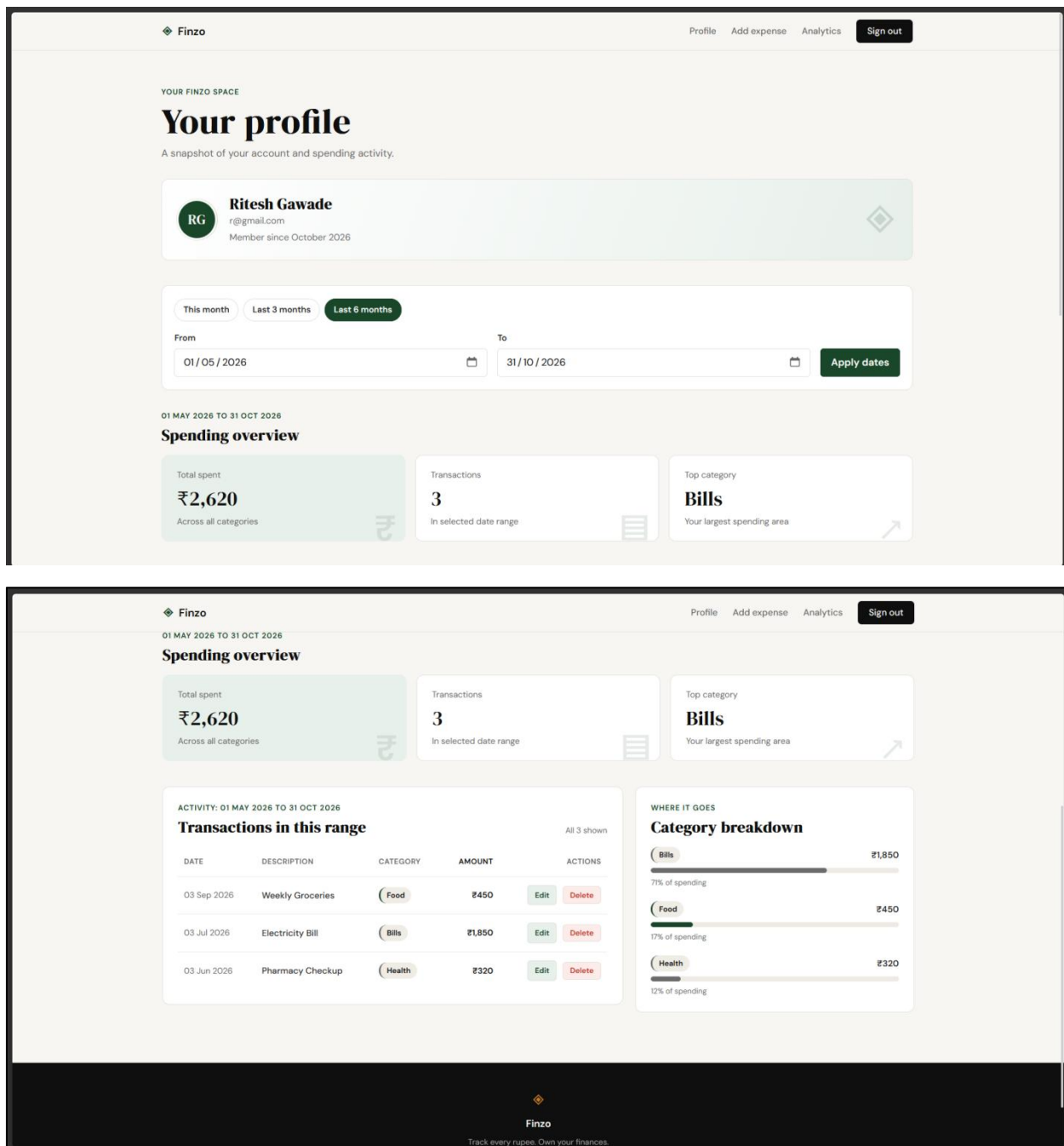
[Figure 2: Registration.]

The form takes full name, email and password. The schema requires a password of at least eight characters.

The screenshot shows the 'Welcome back' page of the Finzo application. The header is identical to the registration page, with the Finzo logo and 'Sign in' and 'Get started' links. The main heading is 'Welcome back' with the subtext 'Sign in to your Finzo account'. The login form is centered and contains two input fields: 'Email address' (filled with 'r@gmail.com') and 'Password' (filled with eight dots). Below these fields is a black 'Sign in' button. At the bottom of the form, there is a link that says 'Don't have an account? Create one free'. The footer features the Finzo logo.

[Figure 3: Login.]

After sign-in, the navigation changes to Profile, Add expense, Analytics and Sign out. Logout is a POST action.



[Figure 4: Profile overview and date filter.]

The profile shows the user's name, email and join month, the date presets (This month, Last 3 months, Last 6 months) and custom From and To dates.

For 01 May 2026 to 31 Oct 2026 it shows total spent **₹2,620** over **3 transactions**, with **Bills** as the top category. The category breakdown (Bills ₹1,850, Food ₹450, Health ₹320) adds up to the total.

Finzo

ProfileAdd expenseAnalyticsSign out

TRACK A NEW SPEND

Add expense

Capture a one-off purchase, recurring bill, or everyday item.

Title

Groceries, rent, flights...

Amount

0.00

Category

Food

Description

Add details about this expense

Save expenseCancel

Finzo

ProfileAdd expenseAnalyticsSign out

UPDATE A SPEND

Edit expense

Adjust the details for this expense and save your changes.

Title

Weekly Groceries

Amount

450.00

Category

Food

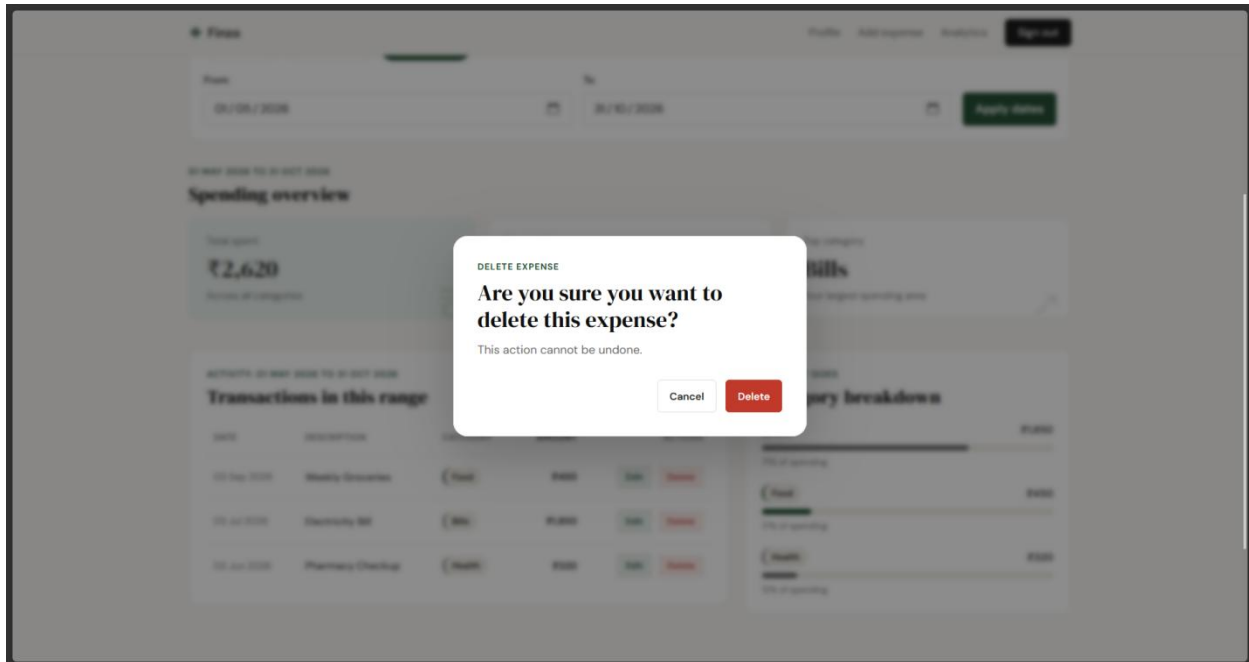
Description

Supermarket run

Update expenseCancel

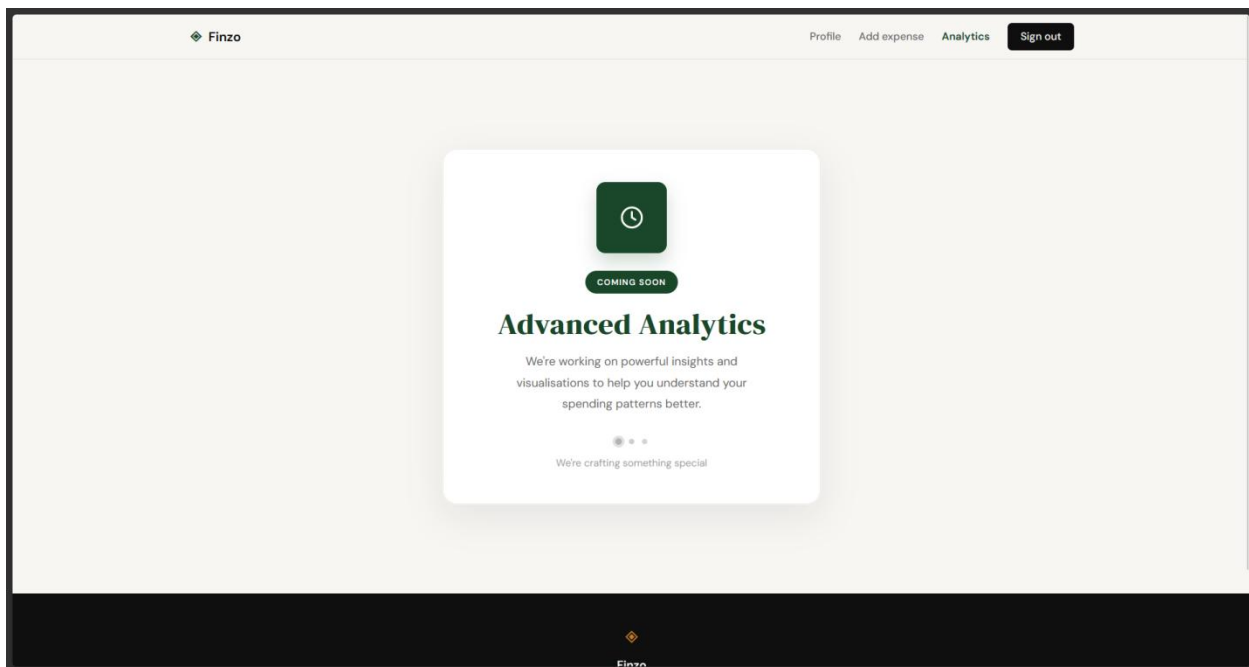
[Figure 5: Add and edit expense.]

The add form has title, amount, category and description. The edit form opens with the saved values, for example Weekly Groceries, 450.00, Food.



[Figure 6: Delete confirmation.]

The dialog asks for confirmation and warns that the action cannot be undone. Confirming sends a POST request limited to the signed-in user.



[Figure 7: Analytics placeholder.]

The page shows 'Coming Soon' and is not presented as working analytics.

Known limitations

- Analytics is only a placeholder. It does not read expense data or draw charts.
- Two profile tests fail at the last run. Their stubbed transactions have no id, and `templates/profile.html` needs it to build the Edit link, so Starlette raises `AssertionError: Must not be empty`.
- The failing tests are `test_profile_renders_repository_backed_context` and `test_profile_route_passes_date_query_parameters_to_service`.
- This is a test-fixture mismatch, not a proven defect for real records, because the service adds transaction IDs.
- Tests mock the database. No live MySQL integration result exists.
- No CI workflow is checked in.

2.2 Code Structure

`app.py` creates the application, sets up sessions and templates, includes routers, and serves public and health endpoints.

Route modules handle HTTP and session work. Services normalize data and build page context. Repositories contain explicit MySQL queries. The database/ package owns connections and query execution. `schemas.py` validates form input.

```
.
├── app.py
├── database/      # MySQL connection, query helpers, table contracts
├── dependencies/  # Session authentication helpers
├── repositories/  # Parameterized user and expense SQL
├── routes/        # Auth, profile, expense, analytics (HTTP only)
├── services/      # Auth, expense, profile business logic
├── schemas.py     # Pydantic input validation
├── templates/     # Shared layout and server-rendered pages
├── static/        # CSS and vanilla JavaScript
├── tests/         # Pytest coverage
├── .github/       # agents, plan, prompts, skills, specs
├── requirements.txt
├── Dockerfile
└── .env.example
```

The expense and profile features use deep modules. The route handles HTTP only, while the service and repository hide validation and SQL behind small functions.

Health and public pages stay in `app.py`, so that part is shallower. There is no ORM or migration framework.

2.3 Refactoring

Example 1: Move the profile route and data access out of `app.py` (commit 09c6641)

- **Before:** The `/profile` handler was inside `app.py`. No profile service or profile repository existed.
- **Problem:** The entry point mixed page routing with profile behavior.
- **After:** The commit added `routes/profile.py`, a profile service and repository-backed data handling. The route passes the user ID to the service and renders the returned context.

- **Why it is better:** HTTP handling, formatting and SQL each have their own place. This means they can be tested and changed separately.

Example 2: Generalize profile queries for date ranges (commit 48bc132)

- **Before:** The repository had a month-only `get_monthly_category_totals` and a recent-transactions query without date bounds.
- **Problem:** The data layer could not express the date range selected in the interface.
- **After:** The function became `get_category_totals_for_range`, the transaction query gained start and end bounds, and the service gained helpers for date ranges and presets. The route accepts `start_date` and `end_date`.
- **Why it is better:** The data contract now matches the filter in the UI and uses half-open timestamp bounds.

2.4 TDD

The tests are feature-based pytest modules, such as `test_auth_login.py`, `test_database_layer.py`, `test_profile.py`, and numbered tests for date filtering, Analytics, add, edit, delete and pre-deployment checks.

They use FastAPI `TestClient` with mocks for repository and database boundaries. They check redirects, rendered forms, validation, user ownership, parameterized SQL and health responses.

The pre-deployment tests also read the Dockerfile and `.dockerignore`; they do not prove that an image was built.

The `test-feature` prompt asks a test-writer agent to derive tests from the spec. Then a test-runner agent runs only that test and diagnoses failures without changing code.

Observed result: `.\.venv\Scripts\python.exe -m pytest -q --tb=short` gave **95 passed, 2 failed, 2 warnings**. Both failures are the profile tests described in section 2.1. The result is for the current working tree, not for a committed baseline.

```
(.venv) PS C:\Users\Ritesh gawade\OneDrive\Desktop\MSC AI\expense-tracker\expense-tracker> .\.venv\Scripts\python.exe -m pytest -q --tb=short

Read more about it in the
[FastAPI docs for Lifespan Events](https://fastapi.tiangolo.com/advanced/events/).

return self.router.on_event(event_type) # ty: ignore[deprecated]

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== short test summary info =====
FAILED tests/test_profile.py::test_profile_renders_repository_backed_context - AssertionError: Must not be empty
FAILED tests/test_profile.py::test_profile_route_passes_date_query_parameters_to_service - AssertionError: Must not be empty
2 failed, 95 passed, 2 warnings in 3.22s
(.venv) PS C:\Users\Ritesh gawade\OneDrive\Desktop\MSC AI\expense-tracker\expense-tracker> 
```

[Figure 10: Pytest run and failure summary.]

2.5 Maintainability

- **Separation of concerns:** Routes, services, repositories, schemas and database helpers are named by responsibility.
- **Security by design:** SQL is parameterized, and expense reads and writes are limited to the signed-in user.
- **Configuration:** `DATABASE_URL`, `SESSION_SECRET_KEY` and the cookie security flag come from the environment. `.env.example` lists names only, and `.gitignore` excludes `.env` and `.env.docker`.
- **Error handling and logging:** Database and authentication failures are logged at the application boundary, and users see generic messages.
- **DRY:** `base.html` provides navigation, footer and page blocks. CSS variables, breakpoints and form styles are shared.
- **Documentation:** Specs state routes, data changes, files, constraints and a definition of done. Copilot instructions, agents and prompts record the rules.
- **Open gaps:** The two failing tests lower confidence until they are fixed, and there is no CI workflow.

3. Prompting Strategies for Build Quality

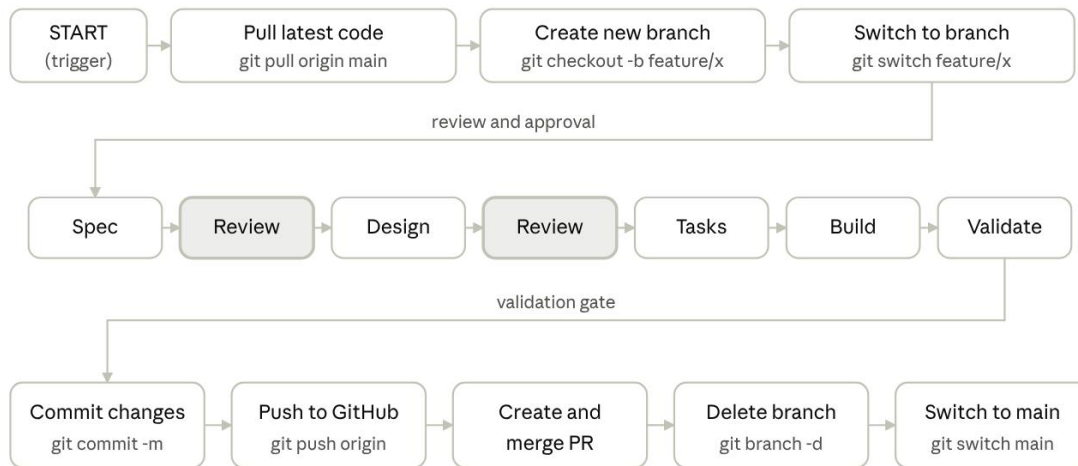
3.1 Prompting Strategies

- **Project-context rules:** `.github/copilot-instructions.md` fixes the stack (FastAPI, MySQL, PyMySQL, Jinja2), the layered design, user ownership, secret handling and parameterized SQL.
- **Spec-first features:** `create-spec.md` tells the copilot to inspect the project and write a numbered spec with routes, data changes, files, constraints, exclusions and a definition of done.
- **Constrained implementation:** Specs require reuse of existing auth, templates and database helpers, and forbid extra dependencies and ORMs.
- **Tests from the spec:** `test-feature.md` makes a test-writer agent derive tests from the spec, then hands the file to a test-runner agent. It forbids silent code changes during diagnosis and use of a live production database.
- **Parallel review:** `code-review-feature.md` runs a security reviewer and a quality reviewer on a scoped diff against the spec, then asks permission before applying findings.
- **Reusable UI context:** The `frontend-design` skill tells the copilot to inspect the real project and reuse the shared layout, CSS tokens and auth flow.
- **Development data:** `seed-user.md` checks email uniqueness before insert. `seed-expense.md` needs an existing user, parameterized SQL and one transaction with rollback.
- **Deployment limits:** Spec 11 defines a platform-neutral pytest and Docker check with mocked database access and no publishing.

3.2 Spec-Driven Development Flow

Every feature follows the spec-driven development (SDD) flow in Figure 11. Git steps wrap the work, so each feature has its own branch and pull request.

Each feature passes a review gate and a validation gate before merge



Shaded boxes are review steps. Work moves on only after approval.

Figure 11: Spec-driven development (SDD) flow.

The flow contains git setup, 7 phases, 2 gates, and git finish.

Spec and design are each reviewed and approved before the next step. The validation gate must pass before commit, push and merge.

3.3 Prompt-to-Practice Mapping

Prompt or file	Practice	Purpose	Evidence
Copilot project instructions	Architecture and security rules	Keep features on direct MySQL access, ownership checks, environment secrets and layering	<code>.github/copilot-instructions.md</code>
Feature-spec prompt	Spec, plan, definition of done	Reduce assumptions and set clear acceptance criteria	<code>.github/prompts/create-spec.md</code> ; <code>.github/specs/01-*.md</code> to <code>11-*.md</code>
Test-feature prompt	Spec-derived tests, then isolated test run	Check behavior without copying it from the implementation	<code>.github/prompts/test-feature.md</code> ; <code>finzo-test-writer.md</code> ;

Prompt or file	Practice	Purpose	Evidence
			finzo-test-runner.md
Code-review prompt	Parallel security and quality review	Separate review concerns and collect findings before changes	.github/prompts/code-review-feature.md; finzo-security-reviewer.md; finzo-quality-reviewer.md
Frontend skill	Reuse the existing design system	Avoid parallel CSS, layout and auth code	.github/skills/frontend-design/SKILL.md
Seed-data prompts	Safe test-data creation	Verify the user, use parameterized queries, roll back on failure	seed-user.md; seed-expense.md
Pre-deployment spec	Local tests and container checks	Check configuration and health without deploying	.github/specs/11-pre-deployment-testing.md; Dockerfile

3.4 What Worked / What Did Not

What worked. Specs with clear exclusions and a definition of done kept features small. The copilot instructions kept the stack fixed. There is no ORM, and update and delete queries are limited to the signed-in user. Separate security and quality reviewers gave two views of the same diff.

What did not work. Both seed prompts tell the copilot to read `database/db.py`, but this file does not exist. The database module is `database/__init__.py`. This is a stale prompt reference, not a runtime bug.

Poor AI output and prompt improvement:

1. **Original prompt:**
“Create a hero section for the Finzo expense-tracking website.”
2. **Poor output:**
The result was mostly text: a headline, a short description, and a call-to-action. It did not give visitors a strong visual idea of what Finzo does.
3. **Why it was poor:**
The prompt described the page topic but did not describe the visual layout or style. Without a visual reference, the generated hero did not have a clear product-focused composition.
4. **Improved prompt:**
“Use the attached reference image as the visual guide for Finzo’s landing-page hero. Match its overall composition and hierarchy, with a clear headline, concise supporting text, a prominent account-creation button, and a visual that suggests expense tracking. Adapt the design to

Finzo's existing website rather than copying the reference blindly.

Follow `.github/skills/frontend-design/SKILL.md` while implementing the UI: preserve the existing theme, typography, shared layout, and CSS conventions, and make the hero responsive."

5. Final result:

The hero now pairs Finzo's headline and sign-up action with a spending-chart visual and summary cards for monthly spending, budget remaining, and transactions. The frontend-design skill was used during UI generation to help keep the site's theme consistent.

6. Evidence:

Commit `d7100f4` is titled `landing: redesign hero section to match mockup`. The result is visible in the landing-page template and stylesheet. The original prompt, first AI output, and reference image were not saved, so their descriptions above are reconstructed rather than verbatim.

4. Audit and Evaluation Pipeline

4.1 Evaluation Pipeline

The checks run at the validation gate of the SDD flow (Figure 11). The steps are defined by prompts and agents, so their presence does not prove that every step ran for every feature.

1. Copilot instructions and the feature spec define the expected behavior.
2. A plan is written where one exists.
3. The feature is implemented in the existing route, service and repository structure.
4. The test-writer agent derives tests from the spec.
5. The test-runner agent runs only that feature's test and diagnoses failures without editing code.
6. The security reviewer and the quality reviewer run in parallel on the scoped diff.
7. Pre-deployment pytest checks run. A Docker build and smoke test is defined but no result is recorded.
8. A manual final review happens before commit, push and merge.

The pipeline is local and manual. There is no `.github/workflows/` directory, so no automated CI/CD was found.

4.2 Evaluation Criteria

Criterion	How it is applied in Finzo	Evidence
Domain-specific	Totals, category shares and date ranges must be correct, and every query must be limited to the signed-in user	Profile screenshot: ₹2,620 = ₹1,850 + ₹450 + ₹320; ownership tests; parameterized SQL
Generation quality	Code keeps the routes, services and repositories split, uses parameterized SQL and no ORM	Section 2.2; reviewer agents

Criterion	How it is applied in Finzo	Evidence
Instruction following	Output follows the spec and Copilot rules: fixed stack, no extra dependencies, user scope, validation	Specs 01 to 11; spec-based tests and reviews
Cost	The product makes no model calls, so it has no AI cost at runtime. Development used GitHub Copilot through two GitHub accounts, with approximately 50% of the available Copilot usage/credits consumed from each account.	—
Latency	No performance target or benchmark exists. The health endpoint only checks MySQL connectivity.	Section 1.4

4.3 Model Selection

Task	Model used	Reason
Coding	GitHub Copilot — automatic model selection	Copilot automatically selected the model for agentic coding, repository exploration, feature implementation, debugging, and code generation.
Test writing	GPT-5 mini	Used to generate pytest tests from feature specifications, acceptance criteria, and existing project patterns.
Audit and code review	Claude Sonnet	Used for security and code-quality review, including identifying maintainability, architecture, implementation, and potential security issues.
Evaluation and diagnosis	GitHub Copilot — automatic model selection	Copilot automatically selected the model to analyze test results, failures, implementation issues, and assist with diagnosing corrective

Task	Model used	Reason
		actions.

4.4 Broken Windows Found

ID	Problem	Evidence
BW1	Seed prompts point to database/db.py, which does not exist	seed-user.md, seed-expense.md
BW2	Tests mock the database, so no live MySQL result exists	tests/
BW3	Landing page text does not match the product: it mentions 'stay on budget' and 'Budget left' although budgets are out of scope, and it lists 'date' as an input although the Add expense form has no date field	Figures 1 and 5

4.5 Fixes Applied

ID	Status at report time	Action or reason
BW1	Open	Change the path to database/__init__.py in both prompts
BW2	Accepted	Mocks keep tests fast and safe; a live integration test is future work
BW3	Open	Reword the landing text or mark the figures as examples

5. Overall Build Process

5.1 Build Process

Phase	Work	Approx. time
Setup	Copilot instructions, project structure, database layer, configuration	1–2 days

Phase	Work	Approx. time
Authentication	Registration, login, logout, signed sessions	1–2 days
Profile	Profile page, summary, category breakdown, date presets and custom filter	2–3 days
Expenses	Add, edit and delete, with ownership checks	2–3 days
Analytics	Placeholder page	0.5–1 day
Quality and deployment	Reviews, pre-deployment tests, Dockerfile, health check	2–3 days

5.2 Technology Stack

Copy the versions from `requirements.txt` (or `pip freeze`) and the `Dockerfile`.

Layer	Technology
Language	Python
Web framework	FastAPI, Starlette sessions, Uvicorn
Templates	Jinja2, HTML, CSS, vanilla JavaScript
Validation	Pydantic
Password hashing	Werkzeug
Database	MySQL with PyMySQL (direct SQL, no ORM)
Testing	pytest, FastAPI TestClient
Packaging	Docker
Version control	Git and GitHub

5.3 AI Copilot Used

The project uses GitHub Copilot in VS Code, with agent mode enabled.

This is reflected in the `.github/` directory, which contains `copilot-instructions.md`, agent definitions, prompts, skills, and feature specifications. The VS Code settings also enable `chat.agent`.

Copilot was used to:

- write specifications;
- plan and implement features;
- write and run tests;
- review code for security and quality;
- create seed data.

5.4 Prompt Log

These are the prompts and agent files stored in the repository.

No.	File	Purpose
1	<code>.github/copilot-instructions.md</code>	Project rules: stack, architecture, security, secrets
2	<code>.github/prompts/create-spec.md</code>	Inspect the project and write a numbered feature spec
3	<code>.github/prompts/test-feature.md</code>	Spec-based test writing, then isolated test run
4	<code>.github/agents/finzo-test-writer.md</code>	Derive tests from the spec
5	<code>.github/agents/finzo-test-runner.md</code>	Run one feature test and diagnose without editing code
6	<code>.github/prompts/code-review-feature.md</code>	Run both reviewers on a scoped diff
7	<code>.github/agents/finzo-security-reviewer.md</code>	Security review against the spec
8	<code>.github/agents/finzo-quality-reviewer.md</code>	Quality review against the spec
9	<code>.github/skills/frontend-design/SKILL.md</code>	Reuse the existing layout, CSS and auth flow
10	<code>.github/prompts/seed-user.md</code>	Create a demo user after an email uniqueness check
11	<code>.github/prompts/seed-expense.md</code>	Create demo expenses in one transaction with rollback
12	<code>.github/specs/11-pre-deployment-testing.md</code>	Pre-deployment test and Docker check

5.5 Summary

Finzo has a working core for personal expense tracking: accounts, sessions, per-user expense add, edit and delete, profile summaries and date filters.

Specs, project rules and review agents guided the AI copilot, and the layered structure keeps responsibilities clear.

The quality gate is not fully green yet: two tests fail, Analytics is a placeholder, and Docker and CI are not verified.